

1 Desenvolvimento e Implementação

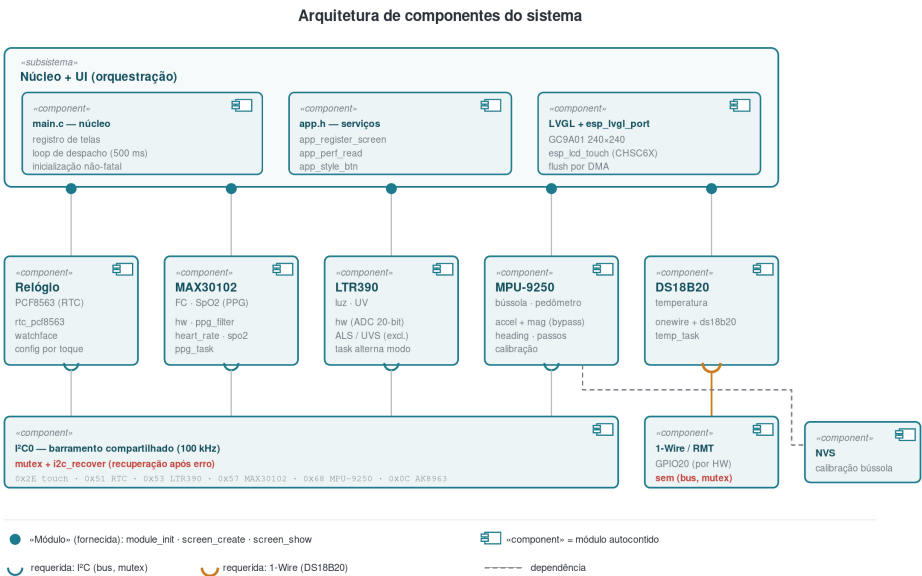
Este capítulo apresenta a implementação da plataforma, desde a organização dos recursos físicos até a integração dos módulos funcionais. A descrição foi conferida diretamente na versão corrente do firmware em `Codigos/IDroid/main`; documentos históricos foram usados apenas para contextualizar decisões de desenvolvimento e não como autoridade sobre a versão integrada.

1.1 Visão geral do sistema

A plataforma foi organizada em quatro níveis: hardware, drivers, serviços compartilhados e módulos funcionais. O hardware reúne o XIAO ESP32-C6, o Round Display e os sensores. Os drivers encapsulam os protocolos e registradores de cada dispositivo. Os serviços coordenam recursos utilizados por mais de um módulo, como I²C, LVGL e NVS. Os módulos associam aquisição, processamento, estado funcional e tela.

A Figura 1 sintetiza os componentes confirmados e as direções de controle, dados e acesso ao hardware.

Figura 1 – Arquitetura de componentes da plataforma implementada



Fonte: Elaborado pelo autor a partir do firmware integrado.

O núcleo não executa os algoritmos de cada sensor. Ele inicializa os serviços, solicita a inicialização dos módulos, cria as telas e despacha a atualização correspondente à tela ativa. A aquisição periódica permanece nas

tarefas dos módulos, enquanto a LVGL é manipulada segundo o mecanismo de exclusão fornecido por sua porta para o ESP-IDF.

1.2 Arquitetura de hardware

1.2.1 Barramento I²C compartilhado

O I²C foi criado uma única vez pelo núcleo com a API mestre introduzida nas versões recentes do ESP-IDF. O barramento opera a 100 kHz em SDA GPIO22 e SCL GPIO23. Cada driver registra seu dispositivo lógico nessa instância, em vez de criar um controlador próprio.

Seis endereços coexistem no barramento: CHSC6X em 0x2E, PCF8563 em 0x51, LTR390 em 0x53, MAX30102 em 0x57, MPU-9250 em 0x68 e AK8963 em 0x0C. O último torna-se acessível externamente quando o MPU-9250 é colocado em modo *bypass*. Embora pertençam ao mesmo encapsulamento funcional, MPU-9250 e AK8963 constituem alvos I²C distintos.

A frequência de 100 kHz foi adotada na integração, substituindo configurações mais altas usadas em testes isolados. A redução aumenta a margem para os tempos de subida na montagem com fios e múltiplas placas. Como todos os dispositivos compartilham o controlador e as linhas, a coordenação por software e a recuperação do barramento foram tratadas como serviços sistêmicos.

1.2.2 Interface SPI

O display GC9A01A utiliza o periférico SPI2 a 40 MHz. As linhas de clock e dados são compartilháveis com o slot microSD do Round Display, enquanto sinais de seleção independentes delimitam as transações. O firmware integrado utiliza o controlador de LCD disponibilizado como componente do ecossistema ESP-IDF; o cartão não integra as funções avaliadas.

As transferências gráficas são predominantemente unidirecionais, do ESP32-C6 para o controlador. O uso de DMA pela porta gráfica permite transferir blocos de pixels sem movimentação individual pela CPU. O acesso ao display permanece encapsulado pelo driver e pela LVGL, de modo que módulos funcionais criam objetos gráficos sem iniciar transações SPI diretamente.

1.2.3 Interface 1-Wire/RMT

O DS18B20 utiliza uma interface separada, no GPIO20, com resistor de *pull-up* externo. Os intervalos do protocolo 1-Wire são produzidos e amostrados por meio do periférico RMT. Essa solução reduz a dependência de atrasos

ativos em software e evita que a temporização de cada bit dependa diretamente da execução contínua de uma tarefa.

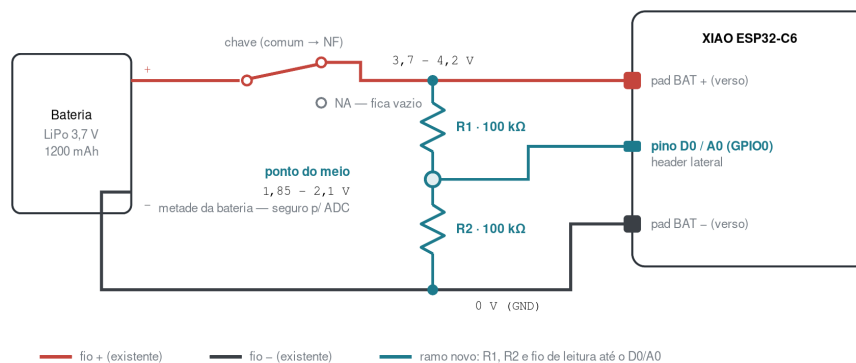
Foram reutilizados os componentes oficiais `onewire_bus` e `ds18b20`. O módulo de temperatura recebe valores já convertidos pela API do componente e não conhece a representação dos símbolos do RMT. A separação demonstra que o contrato funcional não exige que todos os módulos utilizem o mesmo transporte.

1.2.4 Alimentação

O protótipo pode ser alimentado por USB por meio do XIAO ESP32-C6 ou por bateria conectada ao circuito do Round Display. A versão implementada não inclui gerenciamento avançado de energia, e o backlight permanece ligado durante a operação. Recursos de suspensão profunda, controle de brilho e otimização de autonomia não fazem parte do desenvolvimento descrito.

A leitura da tensão da bateria também integra a versão final. O positivo após a chave alimenta um divisor 1:2 formado por dois resistores de 100 k Ω ; o ponto médio segue para o GPIO0/A0 do XIAO ESP32-C6. O firmware aplica calibração do ADC, média de 16 amostras e suavização temporal antes de apresentar o estado estimado. A Figura 2 mostra o circuito efetivamente empregado. Como a estimativa se baseia em tensão, ela não equivale a um contador de carga.

Figura 2 – Divisor resistivo empregado na leitura da tensão da bateria



Fonte: Elaborado pelo autor a partir do circuito implementado e de `bateria.c`.

Durante a integração, a alimentação do MAX30102 exigiu atenção por causa dos pulsos de corrente dos emissores. Os testes A/B registrados no próprio código associaram a configuração anterior, de aproximadamente 14,2 mA por emissor, a afundamento da linha de 3,3 V e travamento do I²C na placa final. A configuração integrada passou a empregar o valor 0x24, correspondente a aproximadamente 7,2 mA por emissor.

1.2.5 Placa protótipo e invólucro

A integração deixou a protoboard de desenvolvimento e foi transferida para uma placa protótipo autoral, cujo esquema, leiaute, renderizações e registro fotográfico foram preservados. Um invólucro também foi fabricado por impressão 3D para acomodar o conjunto. Esses elementos pertencem à implementação realizada; revisões futuras podem aprimorá-los, mas não correspondem a atividades pendentes de construção.

1.3 Arquitetura de software

1.3.1 Organização do firmware

O firmware foi dividido por responsabilidade. O núcleo contém o ponto de entrada, os registros da aplicação e os serviços comuns. Os drivers tratam comunicação e configuração dos circuitos integrados. Cada função apresentada ao usuário reúne os arquivos de seu módulo, seu processamento e sua tela. Recursos gráficos reutilizáveis permanecem em uma camada de interface.

No diretório `main`, o arquivo `main.c` concentra inicialização, menus e despacho; `app.h` expõe os serviços comuns da aplicação; `i2c_recover.c` encapsula a recuperação do barramento; `relogio/` contém RTC, configuração, bateria e tela principal; `sensores/` separa os módulos por dispositivo; e `ui/` preserva a base gráfica exportada. Dentro dos sensores, os sufixos `_hw`, `_process` e `_screen` materializam, quando aplicável, a separação entre acesso ao dispositivo, tratamento e apresentação.

O aspecto arquitetural relevante é a direção das dependências: os módulos utilizam serviços e drivers; o núcleo utiliza suas interfaces públicas; e um módulo funcional não acessa diretamente o estado interno de outro.

1.3.2 Contrato uniforme de módulo

O contrato estabelece três etapas comuns: inicializar a função, criar sua tela e solicitar sua exibição. Em forma abstrata, ele pode ser representado por:

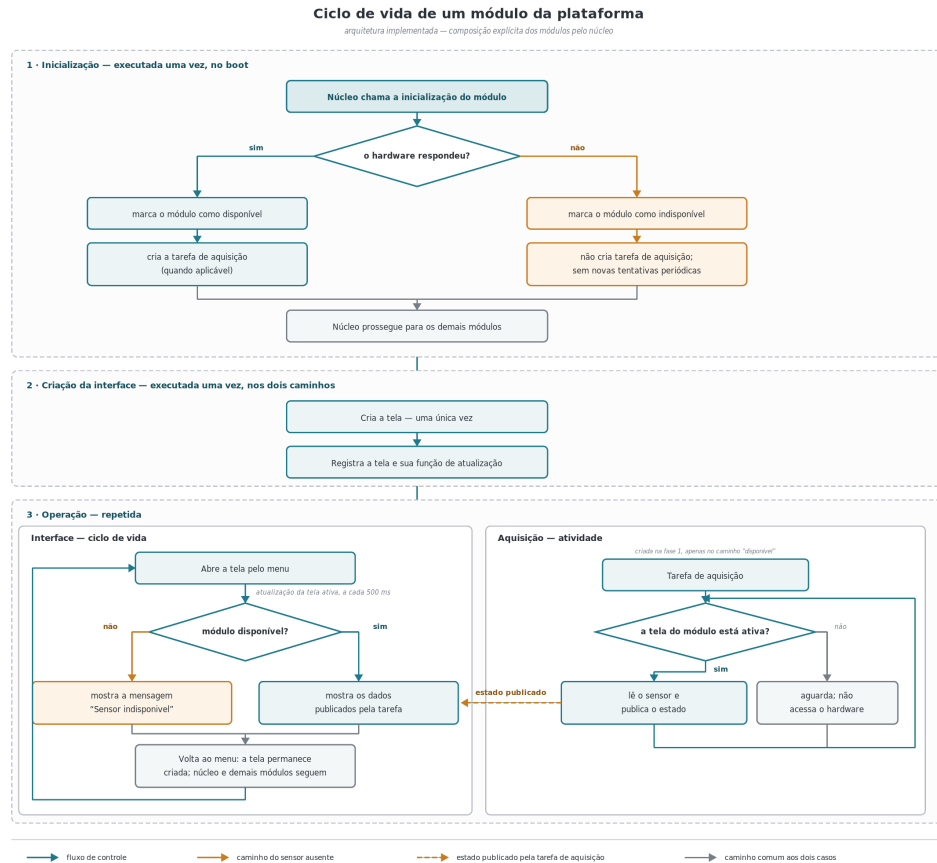
```
[c]
esp_err_t modulo_init(/* depend\^encias necess\'arias */);
void modulo_screen_create(lv_obj_t *menu_origem);
void modulo_screen_show(void);
```

A inicialização prepara o driver, o estado e, quando necessário, a tarefa de aquisição. A criação constrói uma única vez os objetos LVGL e registra a função que atualizará a tela. A exibição solicita a navegação para a tela já criada.

A uniformidade refere-se ao ciclo de vida e às responsabilidades observáveis, não à exigência de assinaturas literalmente idênticas. Módulos I²C recebem a referência do barramento e do mutex; o DS18B20 cria sua interface 1-Wire sem essas dependências. Essa variação conserva o transporte dentro do módulo sem obrigar o núcleo a simular uma dependência inexistente.

A Figura 3 mostra como o mesmo ciclo preserva uma tela informativa tanto no caminho disponível quanto no caminho de sensor ausente.

Figura 3 – Ciclo de vida dos módulos funcionais



Fonte: Elaborado pelo autor a partir do contrato observado no firmware.

1.3.3 Serviços do núcleo

O registro de telas associa um objeto LVGL a uma função de atualização. A cada 500 ms, o despachante consulta a tela ativa e chama somente a função registrada para ela. Menus estáticos não precisam de atualização periódica, enquanto telas de sensores convertem o estado publicado pelas tarefas em texto, indicadores ou gráficos.

O núcleo também concentra a criação do barramento I²C, o scanner de endereços, a recuperação após falhas e os estilos compartilhados da interface. O serviço `app_perf_read()` fornece uma estimativa exploratória de ociosidade e o heap livre, mas, sem uma série controlada, esses valores não foram usados como caracterização de desempenho.

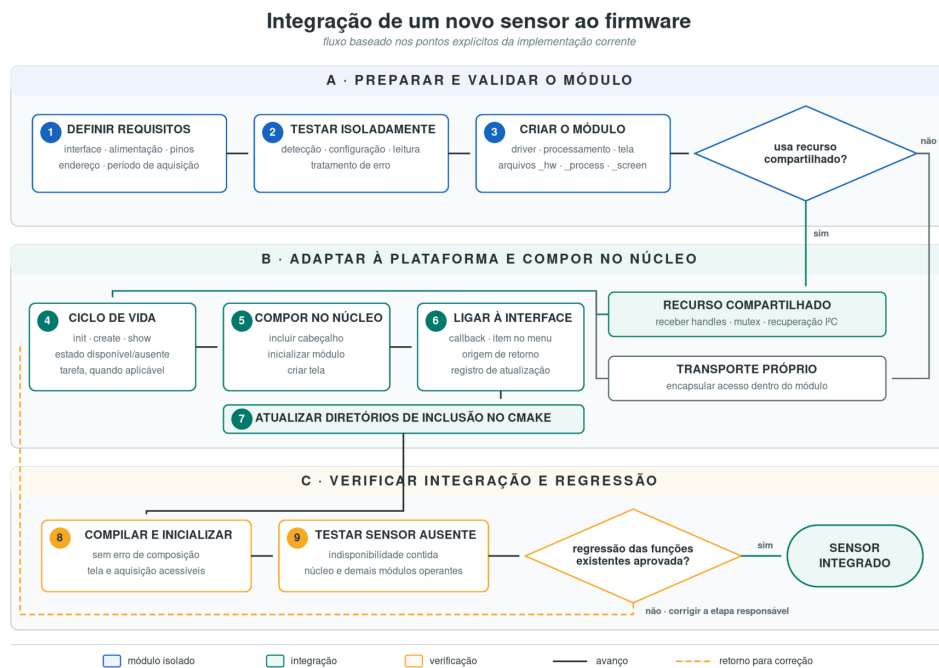
1.3.4 Ciclo de vida dos módulos

Após a inicialização dos serviços básicos, o núcleo percorre os módulos conhecidos. Cada módulo tenta identificar e configurar seu hardware e registra internamente se ficou disponível. A criação das telas ocorre mesmo quando um sensor falha, pois a interface precisa apresentar o estado de indisponibilidade sem depender de nova tentativa contínua.

As tarefas são criadas somente quando necessárias ao módulo. Durante a operação, elas alternam entre bloqueio temporizado, aquisição e publicação do estado. A tela consome esse estado sem executar diretamente a transação sensorial, evitando que uma atualização gráfica aguarde uma conversão lenta.

Como a composição dos módulos permanece explícita na implementação corrente, a inclusão de um novo sensor exige tanto a preparação do módulo quanto seu registro no núcleo e na interface. A Figura 4 organiza esse procedimento a partir dos pontos confirmados no firmware.

Figura 4 – Fluxo para integração de um novo sensor ao firmware



Fonte: Elaborado pelo autor a partir de `main.c`, `app.h`, `CMakeLists.txt` e da organização dos módulos sensoriais.

1.3.5 Inicialização não fatal

A inicialização de um sensor retorna erro ao núcleo, mas esse retorno não aciona encerramento deliberado da aplicação. O módulo marca sua disponi-

bilidade como falsa, preserva uma mensagem apropriada para a tela e não inicia aquisições que dependam do dispositivo ausente.

O núcleo prossegue com os módulos seguintes e com a criação da interface. Assim, relógio, toque e sensores presentes podem permanecer disponíveis quando um sensor externo não responde. A falha de recursos compartilhados, como o próprio controlador I²C ou o display, possui alcance maior e precisa ser distinguida da ausência isolada de um módulo.

1.3.6 Modelo de concorrência

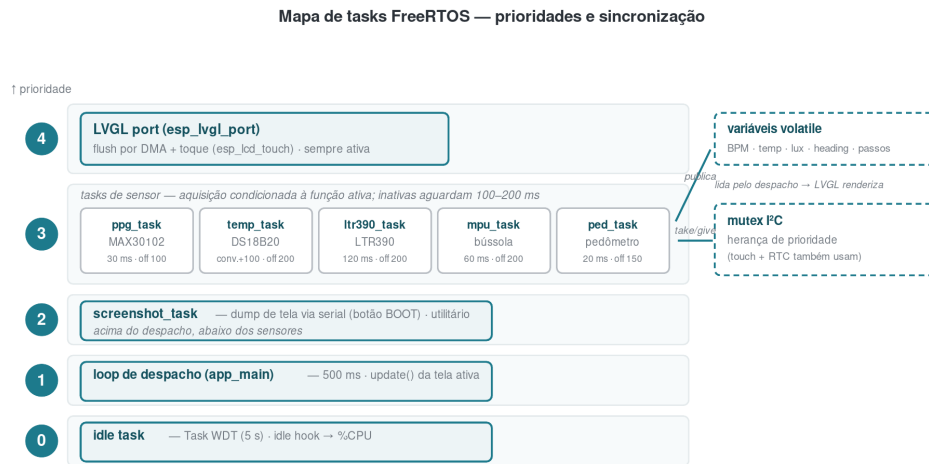
O FreeRTOS opera em configuração de núcleo único. A configuração padrão usada por `esp_lvgl_port` cria a tarefa gráfica com prioridade 4. As tarefas de MAX30102, DS18B20, LTR390, bússola e pedômetro são criadas explicitamente com prioridade 3; a tarefa auxiliar de captura usa prioridade 2; o laço de `app_main` permanece na tarefa principal do ESP-IDF; e a tarefa ociosa ocupa a prioridade 0.

A frequência de *tick* foi configurada em 100 Hz, correspondendo a uma resolução de 10 ms. As tarefas aguardam com primitivas de bloqueio e não realizam espera ocupada entre aquisições. O watchdog de tarefas supervisiona a capacidade de a tarefa ociosa voltar a executar.

As tarefas sensoriais verificam se sua tela está ativa antes de acessar o hardware. Quando a função não está em uso, elas bloqueiam por um intervalo e voltam a verificar o estado. Esse condicionamento reduz transações e processamento sem suspender ou destruir repetidamente as tarefas.

A Figura 5 organiza as tarefas confirmadas, suas prioridades e os recursos compartilhados.

Figura 5 – Tarefas FreeRTOS e recursos compartilhados no núcleo único



Fonte: Elaborado pelo autor a partir do firmware e da configuração do ESP-IDF.

1.3.7 Comunicação entre tarefas e interface

As tarefas publicam as últimas medidas válidas e indicadores de estado em variáveis pertencentes ao módulo. A função de atualização da tela lê uma fotografia desses valores sob a proteção adequada ao tipo de dado. Variáveis escalares alinhadas de até 32 bits podem ser lidas atômica e pelo processador, mas conjuntos de valores correlacionados exigem cuidado para que a tela não combine amostras de instantes diferentes.

Apenas o contexto autorizado pela porta da LVGL altera objetos gráficos. Quando uma função externa ao contexto gráfico precisa atualizar a interface, ela obtém o bloqueio fornecido pela integração do LVGL. A aquisição permanece independente desse bloqueio para não reter o recurso gráfico durante transações ou processamento.

1.3.8 Inclusão de novo módulo

A inclusão de uma nova função segue uma sequência definida: implementar ou selecionar o driver; criar o estado e a tarefa, quando necessários; implementar as três operações do ciclo de vida; registrar a tela e sua atualização; adicionar o acesso à navegação. Serviços novos somente devem ser acrescentados ao núcleo se forem compartilhados por mais de um módulo ou representarem uma responsabilidade global.

Esse procedimento descreve o ponto de extensão implementado, mas não quantifica seu custo. O número de arquivos alterados, as mudanças no nú-

cleo e o efeito sobre módulos existentes serão determinados pelo ensaio de extensibilidade.

1.4 Robustez do barramento I²C

1.4.1 Problema do NACK

Durante a integração, um NACK deixou de ser apenas um erro local de leitura. Como os dispositivos compartilham controlador e linhas, uma transação interrompida podia afetar a operação seguinte, ainda que destinada a outro endereço. Os documentos de desenvolvimento registram ocorrências em que o controlador retornou estado inválido depois da falha inicial.

Cada driver passou a tratar o retorno da API em todas as transações. A ausência de ACK não é interpretada como dado nulo nem ignorada. O módulo recebe o erro, evita publicar uma amostra inválida e aciona o procedimento comum de recuperação antes de uma tentativa posterior.

1.4.2 Mutex com herança de prioridade

Um mutex único protege as transações do barramento. A tarefa o obtém antes da sequência que deve permanecer indivisível, executa o acesso e o libera em todos os caminhos de saída. A proteção abrange a transação completa, incluindo seleção de registrador e leitura subsequente quando essas etapas formam uma operação lógica.

Foi utilizado mutex, e não semáforo binário, para dispor da herança de prioridade oferecida pelo FreeRTOS. Se uma tarefa prioritária aguarda o barramento retido por outra de menor prioridade, a tarefa proprietária pode herdar temporariamente a prioridade até concluir a região protegida.

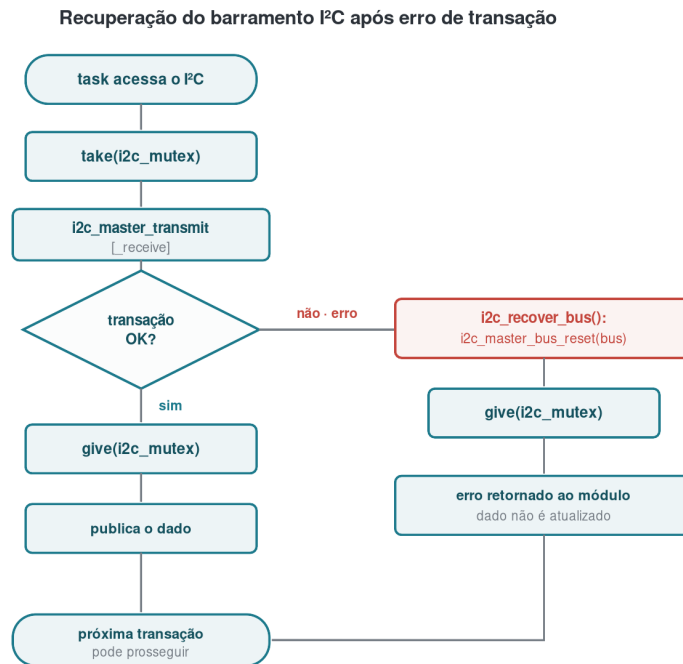
1.4.3 Recuperação do barramento

O serviço de recuperação encapsula a chamada `i2c_master_bus_reset()` oferecida pela API mestre do ESP-IDF (ESPRESSIF SYSTEMS, 2025g). Quando uma transação falha, o driver solicita a recuperação ainda dentro do fluxo controlado e somente depois libera o mutex. Dessa forma, outra tarefa não inicia uma transação enquanto o controlador permanece no estado associado à falha anterior.

O reset não garante que um dispositivo ausente volte a responder. Sua finalidade é devolver o controlador e as linhas a uma condição na qual os demais dispositivos possam ser acessados. O número de tentativas permanece limitado para evitar um ciclo indefinido de falha e recuperação.

A ordem do caminho normal e da recuperação pós-NACK é apresentada na Figura 6.

Figura 6 – Transação I²C e recuperação após falha



Fonte: Elaborado pelo autor a partir de `i2c_recover.c` e dos drivers integrados.

1.4.4 Scanner no boot

Após criar o barramento, o firmware percorre os endereços possíveis e registra os dispositivos que respondem. O scanner foi usado como diagnóstico da montagem e permitiu distinguir endereço incorreto, sensor ausente e problema geral das linhas.

O resultado da varredura não substitui a inicialização dos drivers. Responder ao endereço demonstra presença no barramento, mas não confirma identidade, configuração nem validade dos dados.

1.4.5 Redução das transações do touch

O controlador CHSC6X fornece um sinal de interrupção associado ao toque. A primeira integração consultava o dispositivo por I²C mesmo quando não havia interação. O driver foi alterado para verificar o GPIO17 e iniciar a leitura somente quando o sinal indica evento ativo.

Essa decisão reduz o uso do barramento por uma função que, na maior parte do tempo, permanece ociosa. A quantificação da redução pertence aos resultados; no desenvolvimento, o aspecto relevante é que a interface passou a utilizar uma indicação física de disponibilidade antes da transação.

1.4.6 Efeito dos logs no sistema

Mensagens repetidas do driver I²C durante uma sequência de erros aumentavam o trabalho de formatação e transmissão serial. Em um sistema de núcleo único, essa atividade concorria com a interface e com tarefas temporizadas. Os níveis de log dos componentes mais verbosos foram reduzidos na operação normal, preservando mensagens resumidas e contadores de falha.

Logs detalhados continuam úteis em versões de diagnóstico. Sua ativação deve ser tratada como instrumentação capaz de alterar o comportamento temporal, especialmente durante ensaios de CPU, boot e estabilidade.

1.5 Interface gráfica e relógio

1.5.1 Inicialização do display

O SPI2 é configurado antes da criação da interface gráfica. O componente do GC9A01A recebe os GPIOs, a frequência de 40 MHz, a resolução e os parâmetros de transformação do painel. Após a sequência de inicialização, a porta do LVGL associa o buffer de desenho à função de transferência do controlador.

O conteúdo foi limitado a uma região útil menor que o retângulo de 240×240 pixels, evitando cortar textos e controles próximos aos cantos invisíveis do painel circular.

1.5.2 Integração com LVGL

O `esp_lvgl_port` coordena o temporizador da biblioteca, a entrada por toque e a transferência das regiões invalidadas. As telas foram construídas com widgets, estilos e fontes, em vez de imagens de quadro completo. Essa estratégia permite alterar valores e indicadores sem armazenar uma imagem integral para cada módulo.

Cada tela é criada uma vez e mantida para reutilização. A navegação troca o objeto exibido, enquanto o registro do núcleo seleciona a rotina de atualização apropriada. Operações que modificam objetos utilizam o bloqueio da porta gráfica.

1.5.3 Correção de cores

A representação RGB565 exigiu alinhar a ordem dos bytes produzidos pelo LVGL, a ordem dos elementos de cor e a inversão configurada no painel. A configuração consolidada combina troca dos bytes de 16 bits, ordem BGR e inversão de cor do controlador.

Esses parâmetros atuam em níveis diferentes e não devem ser atribuídos somente ao *endianness* do processador. No firmware corrente, `main.c` define ordem BGR e inversão no controlador, enquanto a opção de troca de bytes do RGB565 permanece na configuração da LVGL.

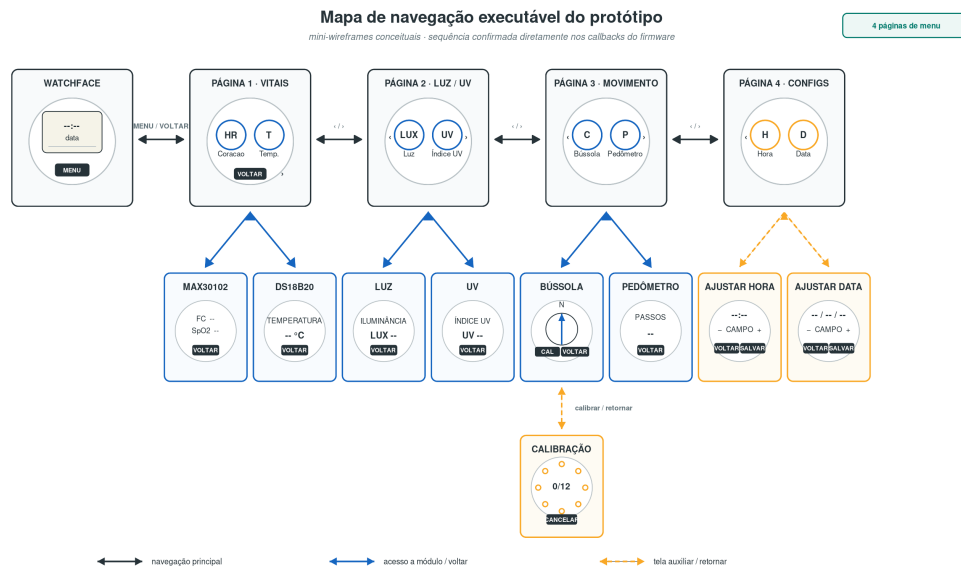
1.5.4 Navegação

A interface utiliza quatro páginas de menu na ordem executável: sinais vitais, com coração e temperatura; luz e UV; movimento, com bússola e pedômetro; e configurações de hora e data. A tela principal abre a primeira página, e as setas laterais percorrem as quatro páginas em sequência.

A navegação é independente dos detalhes do sensor. O item conhece a função pública usada para exibir a tela, mas não acessa dados, registradores nem tarefas do módulo.

A Figura 7 apresenta os acessos aos módulos, os caminhos de retorno e as telas auxiliares.

Figura 7 – Mapa de navegação da interface do protótipo



Fonte: Elaborado pelo autor a partir dos *callbacks* e arquivos de tela do firmware.

1.5.5 Tela principal

A tela principal apresenta hora e data obtidas do PCF8563. Um anel de 60 marcadores distribui a indicação dos segundos ao redor da circunferência. Os elementos centrais permanecem dentro do raio útil e são atualizados sem reconstruir a tela inteira.

O relógio também oferece o ponto de entrada para a configuração de data e hora. A atualização periódica lê os campos necessários do RTC e converte sua representação antes de alterar os rótulos.

1.5.6 Fonte otimizada

A fonte monoespaçada usada na hora foi gerada somente com os algarismos de zero a nove e o caractere de dois-pontos, totalizando 11 glifos. A redução evita incorporar caracteres que não aparecem naquele rótulo.

Como a fonte não contém espaço, o campo ativo na edição não pode ser ocultado por substituição textual. O firmware varia a opacidade do objeto, mantendo a indicação visual sem ampliar o conjunto de glifos.

1.5.7 Configuração por toque

Dois controles na tela permitem avançar o valor atual e confirmar a etapa. Uma máquina de estados percorre hora, minuto, dia, mês e ano. Ao confirmar o último campo, o firmware determina o dia da semana e grava o conjunto no PCF8563.

Durante a edição, o campo ativo alterna sua opacidade para indicar qual valor será alterado. A implementação não depende de botões físicos externos ao Round Display.

1.5.8 RTC PCF8563

O driver próprio lê sete bytes consecutivos a partir do registrador de segundos. Os valores armazenados em BCD são convertidos para inteiros e reunidos em uma estrutura de data e hora. Na escrita, o caminho inverso prepara os campos e preserva os bits de controle definidos pelo dispositivo.

O dia da semana é calculado a partir da data informada antes da gravação. A bateria de reserva do Round Display mantém o RTC durante a interrupção da alimentação principal. O RTC não participa da recuperação de tempo por rede, pois sincronização por NTP não foi implementada.

1.6 Módulo MAX30102

1.6.1 Configuração

O driver verifica a identidade do MAX30102 e programa o modo de aquisição dos canais vermelho e infravermelho. A configuração corrente utiliza taxa de 100 amostras por segundo, resolução de 18 bits, faixa do conversor de 16.384 nA e corrente aproximada de 7,2 mA em cada emissor. Esses valores correspondem, respectivamente, a 0x67 em `SP02_CONFIG` e 0x24 nos registradores de corrente dos LED.

O componente permanece em *shutdown* quando não há medição solicitada. A tela oferece um comando de início, evitando manter os emissores ativos continuamente.

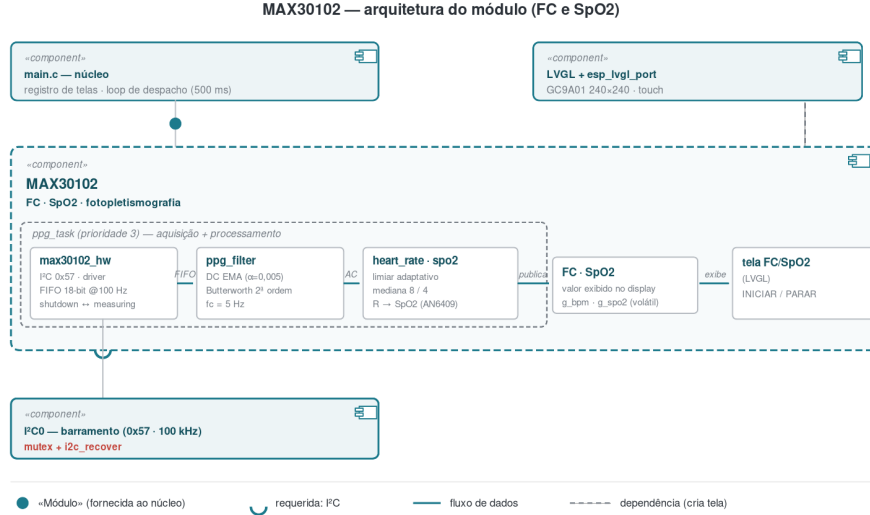
1.6.2 FIFO e aquisição

A FIFO desacopla a produção das amostras do instante exato em que a tarefa obtém o barramento. A tarefa consulta a quantidade disponível, lê os bytes em bloco e recompõe cada valor de 18 bits. Os ponteiros da FIFO e condições de transbordamento são verificados para evitar processar dados incompletos como amostras válidas.

Cada amostra é encaminhada ao detector de contato e, quando aceita, às etapas de condicionamento. A interface consome apenas os valores publicados pelo processamento, sem drenar a FIFO no contexto gráfico.

O fluxo completo, incluindo as portas de aceitação que podem rejeitar uma amostra ou um pulso, é sintetizado na Figura 8.

Figura 8 – Aquisição e processamento implementados no módulo MAX30102



Fonte: Elaborado pelo autor a partir de `max30102_hw.c`, `ppg_filter.c`, `heart_rate.c` e `spo2.c`.

1.6.3 Detecção de dedo

A presença de contato é inferida do nível do sinal óptico bruto com limites distintos para entrada e saída. A histerese reduz alternâncias quando o valor permanece próximo ao limite. Ao perder contato, o módulo invalida as estimativas e reinicializa estados que não devem atravessar duas aquisições independentes.

Essa detecção indica somente que o nível óptico é compatível com contato. Ela não garante qualidade suficiente para estimar frequência cardíaca ou SpO₂.

1.6.4 Filtragem

Uma média móvel exponencial acompanha a componente DC. Durante as primeiras 300 amostras, o coeficiente é 0,1 para acelerar a estabilização; depois, passa a $\alpha = 0,005$. A subtração produz a componente pulsátil, que passa por um filtro Butterworth de segunda ordem com frequência de corte de 5 Hz. A frequência de amostragem usada no cálculo dos coeficientes é 100 Hz.

Os estados dos filtros são preservados entre amostras consecutivas e reiniciados quando começa uma nova sessão ou quando o contato é perdido. O detector ignora as primeiras 500 amostras de cada sessão para que os transientes dos dois estágios decaiam antes de publicar batimentos.

1.6.5 Frequência cardíaca

A detecção utiliza uma diferença com intervalo de quatro amostras, o cruzamento de sua derivada associado a um máximo e um limiar adaptativo de amplitude. Candidatos abaixo de 30% do limiar corrente ou com amplitude absoluta inferior ao limite implementado são rejeitados. Um período refratário de 400 ms impede que oscilações próximas sejam interpretadas como batimentos distintos.

A frequência cardíaca é calculada a partir dos intervalos entre batimentos. Uma mediana sobre oito intervalos reduz a influência de uma detecção isolada. Quando não há quantidade suficiente de eventos válidos, a tela mantém a estimativa como indisponível.

1.6.6 Estimativa de SpO₂

Para cada batimento aceito, o módulo obtém componentes AC e DC dos canais vermelho e infravermelho e calcula a razão de razões. A conversão consolidada utiliza a relação empírica

$$\widehat{\text{SpO}_2} = -45,060R^2 + 30,354R + 94,845.$$

Crítérios de qualidade rejeitam denominadores inadequados, razões fora da faixa admitida e pulsos com baixa modulação. A saída é suavizada pela mediana de quatro estimativas aceitas. O valor permanece denominado estimativa de SpO₂, pois a implementação não passou por validação clínica.

1.6.7 Inversão dos canais em placas de conexão

Durante a depuração, a ordem observada dos canais em determinadas placas de conexão não correspondeu à atribuição inicialmente presumida. Os três primeiros bytes de cada amostra foram tratados como infravermelho e os três seguintes como vermelho na montagem utilizada.

Essa correção pertence à associação entre o componente e a placa específica, não a uma inversão universal do MAX30102. O driver deve registrar ou encapsular a ordem adotada para impedir que o processamento receba canais trocados.

1.6.8 Integração com tarefa e tela

A tarefa do módulo permanece bloqueada quando a tela não está ativa ou quando a medição não foi iniciada. Durante uma sessão, ela obtém o mutex, drena a FIFO, libera o barramento e executa o processamento. A tela apresenta estado de contato, frequência cardíaca e estimativa de SpO₂.

Falhas de comunicação invalidam a aquisição corrente e acionam a recuperação comum do I²C. A interface continua navegável e diferencia sensor indisponível, ausência de contato e estimativa ainda não estabilizada.

1.7 Módulo DS18B20

1.7.1 Componente oficial e RMT

O módulo utiliza os componentes oficiais do barramento 1-Wire e do DS18B20. A criação do barramento informa o GPIO20 e seleciona o RMT como mecanismo de temporização. O sensor encontrado é associado ao componente de nível superior, que oferece operações de configuração e leitura.

Essa reutilização evita manter no projeto uma segunda implementação dos intervalos do protocolo e da verificação CRC. O módulo permanece responsável pelo ciclo funcional e pela apresentação.

1.7.2 Ciclo de conversão

O sensor foi configurado para 11 bits. A tarefa solicita uma conversão, bloqueia durante 375 ms e depois lê o *scratchpad*. Esse tempo corresponde à resolução selecionada na chamada efetiva de configuração.

A espera ocorre por bloqueio da tarefa, e não por laço ativo. Como a temperatura varia lentamente, uma nova conversão só é solicitada no período definido pelo módulo e quando sua função está ativa.

1.7.3 CRC e formatação

O componente verifica o CRC antes de entregar a temperatura. Leituras com erro ou fora da faixa funcional definida para a aplicação não substituem a última amostra válida. O processamento aplica uma média móvel exponencial com $\alpha = 0,5$ e a tela apresenta o resultado com uma casa decimal.

A configuração do LVGL adotada não oferece formatação direta de ponto flutuante pela função usada nos rótulos. O módulo converte o valor para décimos de grau e separa sinal, parte inteira e parte decimal antes de construir o texto.

1.7.4 Integração com tarefa e tela

A inicialização do DS18B20 não recebe o barramento nem o mutex I²C. Depois de encontrar e configurar o sensor, o módulo cria sua tarefa e publica temperatura e estado. A tela lê esses dados e apresenta indisponibilidade quando a inicialização ou a leitura falha.

1.8 Módulo LTR390

1.8.1 Configuração

O driver adiciona o LTR390 ao barramento no endereço 0x53 e a 100 kHz, verifica sua identificação e programa resolução de 18 bits com integração e taxa de 100 ms. A configuração associa ganho $3\times$ ao canal de luz ambiente e ganho $18\times$ ao canal ultravioleta.

1.8.2 ALS e UVS

Os canais ALS e UVS utilizam o mesmo conversor e não operam simultaneamente. O estado do módulo registra o modo configurado, a disponibilidade de nova amostra e as médias mantidas separadamente para iluminância e UVI.

Uma solicitação da tela de luz seleciona ALS, enquanto a tela de ultravioleta seleciona UVS. Essa decisão permanece interna ao módulo e não modifica o núcleo nem o driver de outro sensor.

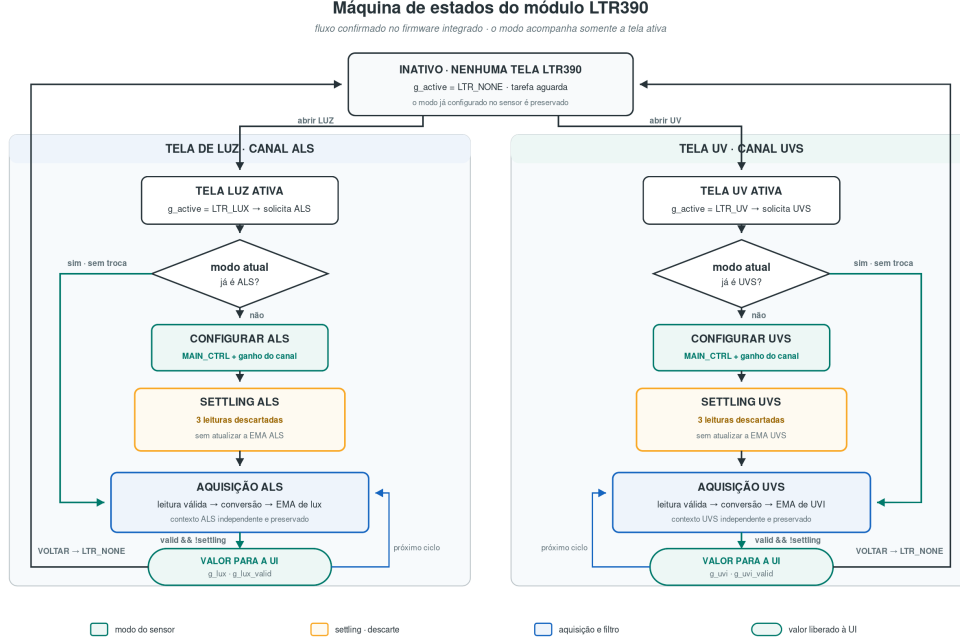
1.8.3 Alternância e estabilização

Quando o modo solicitado difere do atual, o driver reconfigura o sensor e o módulo descarta as três primeiras amostras. O descarte impede que valores produzidos durante a transição sejam publicados como pertencentes ao novo canal.

A tarefa só alterna quando a função ativa exige outro modo. Não há comutação contínua em segundo plano para manter simultaneamente dois valores que não estão sendo exibidos.

A Figura 9 explicita que o modo segue a tela ativa e que os valores somente se tornam válidos para a interface após o descarte das três amostras.

Figura 9 – Máquina de estados implementada para o LTR390



Fonte: Elaborado pelo autor a partir de `ltr390_screen.c` e `ltr390_process.c`.

1.8.4 Conversão para lux e UVI

Na configuração consolidada, a iluminância é calculada por

$$E_v = 0,2 N_{ALS},$$

em que N_{ALS} é a contagem bruta. A estimativa de UVI é calculada por

$$\widehat{UVI} = \frac{N_{UVS}}{575}.$$

Os fatores dependem de ganho e tempo de integração. Por isso, as funções de conversão permanecem associadas à configuração usada pelo driver e deverão ser atualizadas em conjunto caso esses parâmetros mudem.

1.8.5 Remoção do SW_RESET

A sequência inicial executava o comando de reinicialização por software. O dispositivo deixava de responder antes de confirmar a escrita, produzindo NACK em uma operação que, no teste isolado, podia parecer apenas uma particularidade local. No barramento compartilhado, esse erro afetava as transações seguintes.

Como todos os registradores necessários já eram programados explicitamente, o comando foi removido da inicialização. A recuperação do I²C continua disponível para falhas reais, mas não é usada para conservar uma operação desnecessária que produz erro previsível.

1.8.6 Integração com tarefa e tela

A tarefa observa qual das telas do módulo está ativa, configura o modo correspondente, aguarda a estabilização e publica o valor convertido. Cada canal mantém seu estado de filtragem por média móvel exponencial, com $\alpha = 0,3$.

Erros de comunicação não são convertidos em zero. A tela distingue indisponibilidade de uma leitura válida de baixa iluminância ou baixa resposta ultravioleta.

1.9 Módulo de bússola

1.9.1 Bypass e inicialização

O driver inicializa o MPU-9250 em 0x68 e habilita o *bypass* para expor o AK8963 em 0x0C ao controlador principal. Em seguida, verifica a identidade do magnetômetro e configura seu modo de aquisição. Todo o fluxo utiliza o mesmo mutex e o mesmo serviço de recuperação dos demais dispositivos.

Na leitura, os registradores de estado são consultados antes dos dados. O registrador ST2 é lido ao final para concluir a sequência e detectar transbordamento.

1.9.2 ASA e conversão

Os coeficientes ASA armazenados de fábrica são lidos no modo apropriado do AK8963. Eles corrigem diferenças de sensibilidade entre os eixos e são aplicados à conversão das contagens para microteslas:

$$m_i = \text{raw}_i \frac{4912}{32760} ASA_i.$$

O ajuste ASA não substitui a calibração contra perturbações da montagem. Após essa conversão, o módulo aplica deslocamento e escala obtidos pelo procedimento executado no dispositivo.

1.9.3 Calibração

A tela de calibração divide o movimento necessário em 12 setores. Durante a rotação do dispositivo, a tarefa acumula mínimos e máximos de cada

eixo e atualiza a indicação de cobertura. Ao concluir os setores, calcula para cada eixo o centro

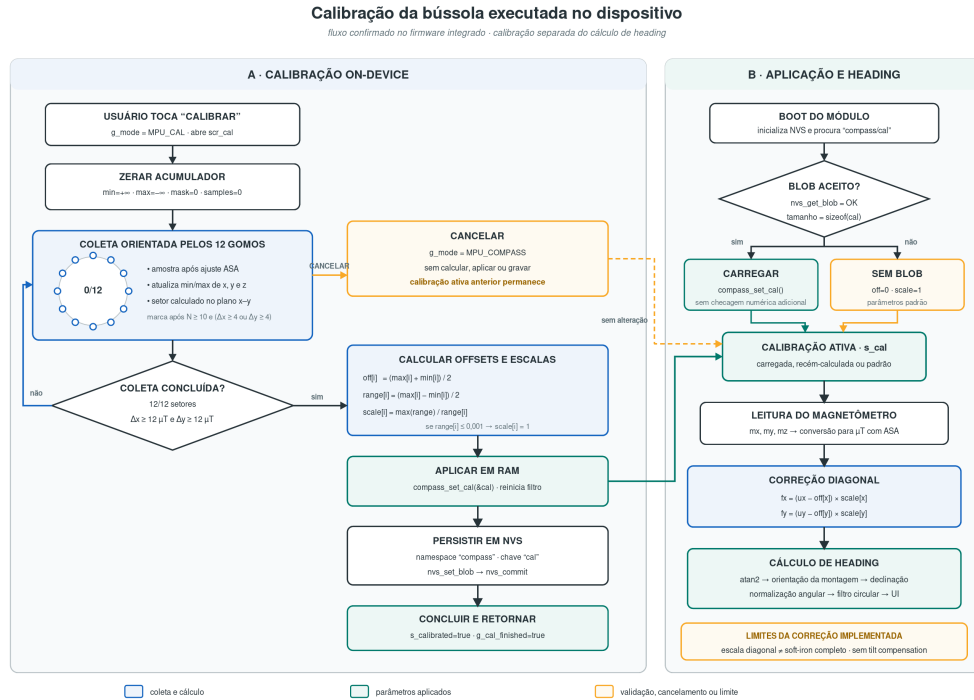
$$b_i = \frac{m_{i,\max} + m_{i,\min}}{2}.$$

e uma escala relativa baseada nas meias-amplitudes observadas. Essa correção representa deslocamento e escala por eixo; não implementa uma transformação elipsoidal completa com termos cruzados.

O procedimento anterior baseado em exportação CSV foi substituído pela calibração executada na própria interface. A versão final não exige recompilar o firmware para aplicar novos parâmetros.

A Figura 10 separa o fluxo acionado pelo usuário do cálculo posterior de *heading* e mostra o caminho de cancelamento que preserva a calibração válida.

Figura 10 – Calibração magnética executada no protótipo



Fonte: Elaborado pelo autor a partir do processamento da bússola, da tela de calibração e da persistência em NVS.

1.9.4 NVS

Offsets, escalas e um indicador de validade são gravados na Non-Volatile Storage (NVS), mecanismo de armazenamento de pares chave-valor em flash fornecido pelo ESP-IDF (ESPRESSIF SYSTEMS, 2025h). Na inicialização, o módulo tenta carregar os valores e utiliza padrões somente quando não há conjunto válido. Uma calibração concluída substitui os dados anteriores de forma controlada.

A persistência é restrita à bússola. O contador do pedômetro não é restaurado após reinicialização.

1.9.5 Heading e filtro

Após aplicar ASA e calibração, o módulo ajusta o mapeamento dos eixos à orientação física da placa e calcula

$$\psi = \text{atan2}(m_y, -m_x).$$

O resultado é normalizado para o intervalo de 0° a 360°. Um filtro com $\alpha = 0,15$ trata a diferença angular pelo caminho mais curto, evitando uma média incorreta entre valores próximos de 359° e 1°.

A implementação inverte o sentido de rotação devido à montagem, acrescenta 180° de orientação física e aplica declinação fixa de -21° , conforme os parâmetros compilados. A tela converte o ângulo nas oito direções cardeais e intercardeais. Não há compensação de inclinação, de modo que a indicação pressupõe o dispositivo aproximadamente nivelado; a declinação fixa também deve ser atualizada para outro local ou época.

1.9.6 Integração com tarefa e tela

A tarefa da bússola e a tarefa do pedômetro são distintas, embora ambas acessem o mesmo MPU-9250 por meio do mutex compartilhado. Quando a bússola está ativa, sua tarefa lê o magnetômetro a cada 60 ms, atualiza o *heading* e responde a comandos de calibração. A tela apresenta ângulo, direção e estado dos parâmetros persistidos.

1.10 Módulo de pedômetro

1.10.1 Tentativa com DMP

A primeira abordagem procurou utilizar o Digital Motion Processor (DMP) do MPU-9250. Ela exigia carregar firmware no dispositivo por registradores

de memória. Na combinação de módulo e implementação testada, esses acessos produziram NACK e impediram concluir a inicialização.

O ocorrido não demonstra incompatibilidade geral do DMP com o ESP32-C6. Diante do custo de investigar uma dependência adicional e da necessidade de controlar o algoritmo, optou-se por um pedômetro implementado no firmware principal.

1.10.2 Algoritmo próprio

O algoritmo utiliza apenas o acelerômetro do MPU-9250. O giroscópio permanece desligado para essa função. As amostras dos três eixos são convertidas para unidades de g , combinadas pela magnitude e processadas por um detector de eventos.

O acesso ao acelerômetro fica na camada de hardware do módulo, enquanto filtragem e contagem permanecem na camada de processamento. A tarefa coordena as duas sem transferir essas responsabilidades ao núcleo.

1.10.3 Configuração

O acelerômetro opera na faixa de $\pm 2g$, com fator de conversão de 16.384 contagens por g , e é amostrado a 50 Hz. O período de 20 ms é compatível com a resolução de 10 ms do *tick* configurado.

1.10.4 Magnitude, EMA, limiar e intervalo mínimo

A magnitude é calculada por

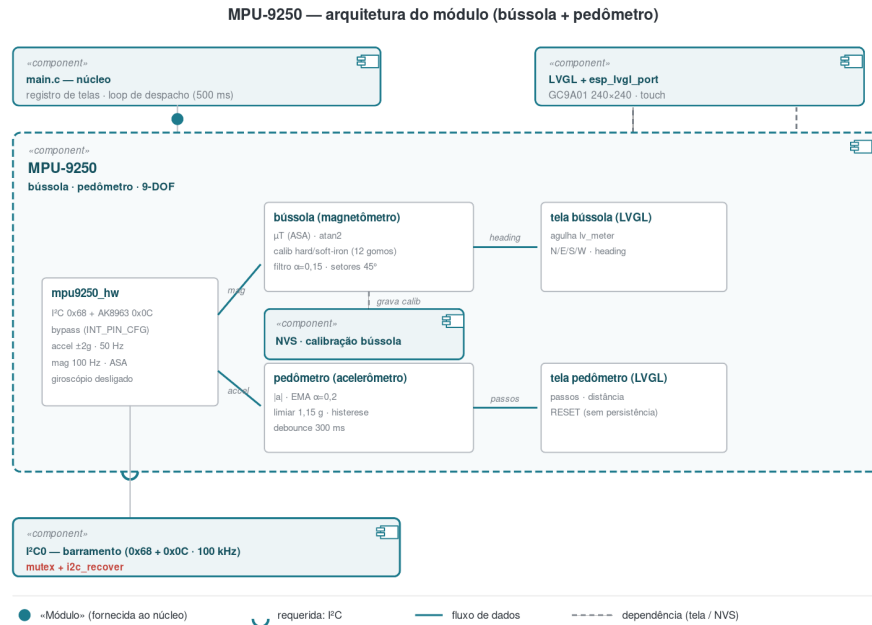
$$a_m = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

Uma média móvel exponencial com $\alpha = 0,2$ reduz componentes rápidas. Um passo é reconhecido quando o sinal cruza o limiar de $1,15 g$, respeitando uma histerese de $0,05 g$. Depois de uma detecção, novos eventos são ignorados por 300 ms, limitando contagens múltiplas do mesmo movimento.

Os parâmetros são fixos e foram escolhidos durante o desenvolvimento. Sua adequação a ritmos e usuários diferentes permanece dependente do ensaio controlado de pedometria.

A arquitetura compartilhada entre bússola e pedômetro é mostrada na Figura 11. O detector implementado segue a relação conceitual entre limiar, histerese e intervalo refratário apresentada na Figura ??, na fundamentação teórica.

Figura 11 – Arquitetura do módulo MPU-9250



Fonte: Elaborado pelo autor a partir de `pedometer_process.c` e `pedometer_screen.c`.

1.10.5 Distância estimada

A distância apresentada é o produto do número de passos por um comprimento fixo de 0,70 m:

$$\hat{d} = 0,70 N_{\text{passos}}.$$

Esse valor não é individualizado por estatura ou padrão de marcha. A interface o identifica como estimativa derivada do contador.

1.10.6 Integração com tarefa e tela

A tarefa coleta aceleração quando a tela do pedômetro está ativa, atualiza o detector e publica passos e distância. A tela oferece comando para zerar a sessão. O estado permanece em memória volátil e retorna a zero após reinicialização.

1.11 Uso de memória e otimizações

A partição de aplicação foi mantida em 1 MiB. A compilação limpa da versão final do binário produziu um arquivo de 784.240 bytes (765,86 KiB),

equivalente a 74,79% da partição. A contribuição incremental de cada módulo ainda não foi medida por compilações controladas.

O pool interno do LVGL foi configurado para 96 KiB. A configuração anterior de 32 KiB não comportava a criação dos objetos da bússola, especialmente o medidor gráfico. Esse ajuste aumenta a memória reservada à interface e precisa ser considerado junto ao heap disponível.

Uma imagem RGB565 de 240×240 pixels requer ao menos 115.200 bytes sem transparência; recursos convertidos podem acrescentar cabeçalho, alinhamento ou outro formato. Para evitar repetir esse custo em cada tela, a interface utiliza widgets, formas e fontes com subconjuntos de glifos.

1.12 Persistência

A NVS armazena os parâmetros de calibração da bússola. O RTC mantém data e hora em seu próprio domínio de alimentação, com bateria de reserva. Esses dois mecanismos possuem ciclos de vida distintos: a NVS conserva parâmetros do firmware, enquanto o PCF8563 mantém a base de calendário.

O pedômetro não possui persistência, e valores instantâneos dos sensores não são gravados. O projeto também não implementa histórico de medições, armazenamento em cartão microSD ou sincronização remota. Essas ausências limitam o escopo do estado persistente e evitam atribuir ao protótipo funções de registro que não foram implementadas.